5

Strings

AIM

Use indexing and built-in string methods to manipulate and process the string values.

LEARNING OUTCOMES

At the end of the chapter, you are expected to

- Access individual characters in a string and apply basic string operations.
- Search and retrieve a substring from a string.
- Use string methods to manipulate strings.

String usage abounds in just about all types of applications. A string consists of a sequence of characters, which includes letters, numbers, punctuation marks and spaces. To represent strings, you can use a single quote, double quotes or triple quotes.

5.1 Creating and Storing Strings

Strings are another basic data type available in Python. They consist of one or more characters surrounded by matching quotation marks. For example,

```
1. >>> single_quote = 'This is a single message'
2. >>> double_quote = "Hey it is my book"
3. >>> single_char_string = "A"
4. >>> empty_string = ""
5. >>> empty_string = ''
6. >>> single_within_double_quote = "Opportunities don't happen. You create
   them."
7. >>> double_within_single_quote = "Why did she call the man 'smart'?"
8. >>> same_quotes = 'I\'ve an idea'
9. >>> triple_quote_string = '''This
   … is
   … triple
   … quote'''
```

```
10. >>> triple_quote_string
    'This\nis\ntriple\nquote'
11. >>> type(single_quote)
    <class 'str'>
```

Strings are surrounded by either single ① or double quotation marks ②. To store a string inside a variable, you just need to assign a string to a variable. In the above code, all the variables on the left side of the assignment operator are string variables. A single character is also treated as string ③. A string does not need to have any characters in it. Both ''④ and "" ⑤ are valid strings, called empty strings. A string enclosed in double quotation marks can contain single quotation marks ⑥. Likewise, a string enclosed in single quotation marks can contain double quotation marks ⑦. So basically, if one type of quotation mark surrounds the string, you have to use the other type within it. If you want to include the same quotation marks within a string as you have used to enclose the string, then you need to preface the inner quote with a backslash ⑧. If you have a string spanning multiple lines, then it can be included within triple quotes ⑨. All white spaces and newlines used inside the triple quotes are literally reflected in the string ⑩. You can find the type of a variable by passing it as an argument to *type()* function. Python strings are of *str* data type ⑪.

5.1.1 The *str()* Function

The *str()* function returns a string which is considered an informal or nicely printable representation of the given object. The syntax for *str()* function is,

str(object)

It returns a string version of the object. If the object is not provided, then it returns an empty string.

```
1. >>> str(10)
   '10'
2. >>> create_string = str()
3. >>> type(create_string)
   <class 'str'>
```

Here integer type is converted to string type. Notice the single quotes to represent the string ①. The *create_string* is an empty string ② of type *str* ③.

5.2 Basic String Operations

In Python, strings can also be concatenated using + sign and * operator is used to create a repeated sequence of strings.

```
1. >>> string_1 = "face"
2. >>> string_2 = "book"
3. >>> concatenated_string = string_1 + string_2
4. >>> concatenated_string
   'facebook'
```

```
5. >>> concatenated_string_with_space = "Hi " + "There"
6. >>> concatenated_string_with_space
   'Hi There'
7. >>> singer = 50 + "cent"
   Traceback (most recent call last):
     File "<stdin>", line 1, in <module>
   TypeError: unsupported operand type(s) for +: 'int' and 'str'
8. >>> singer = str(50) + "cent"
9. >>> singer
   '50cent'
10. >>> repetition_of_string = "wow" * 5
11. >>> repetition_of_string
    'wowwowwowwowwow'
```

Two string variables are assigned with "face"① and "book" ② string values. The *string_1* and *string_2* are concatenated using + operator to form a new string. The new string *concatenated_string* ④ has the values of both the strings ③. As you can see in the output, there is no space between the two concatenated string values. If you need whitespace between concatenated strings ⑥, all you need to do is include whitespace within a string like in ⑤. You cannot use the + operator to concatenate values of two different types. For example, you cannot concatenate string data type with integer data type ⑦. You need to convert integer type to string type and then concatenate the values ⑧–⑨. You can use the multiplication operator * on a string ⑩. It repeats the string the number of times you specify and the string value "wow" is repeated five times ⑪.



Python cannot concatenate string value with integer value since they are of different data types. You need to convert integer type to string type before concatenating integer and string values.

You can check for the presence of a string in another string using *in* and *not in* membership operators. It returns either a Boolean *True* or *False*. The *in* operator evaluates to *True* if the string value in the left operand appears in the sequence of characters of string value in right operand. The *not in* operator evaluates to *True* if the string value in the left operand does not appear in the sequence of characters of string value in right operand.

```
1. >>> fruit_string = "apple is a fruit"
2. >>> fruit_sub_string = "apple"
3. >>> fruit_sub_string in fruit_string
   True
4. >>> another_fruit_string = "orange"
5. >>> another_fruit_string not in fruit_string
   True
```

Statement ③ returns *True* because the string "*apple*" is present in the string "*apple is a fruit*". The *not in* operator evaluates to *True* as the string "*orange*" is not present in "*apple is a fruit*" string ⑤.

5.2.1 String Comparison

You can use (>, <, <=, >=, ==, !=) to compare two strings resulting in either Boolean *True* or *False* value. Python compares strings using ASCII value of the characters. For example,

1. >>> "january" == "jane"
   False
2. >>> "january" != "jane"
   True
3. >>> "january" < "jane"
   False
4. >>> "january" > "jane"
   True
5. >>> "january" <= "jane"
   False
6. >>> "january" >= "jane"
   True
7. >>> "filled" > ""
   True

Strings can be compared using various comparison operators ①–⑦. String equality is compared using == (double equal sign). String inequality is compared using != sign. Suppose you have string_1 as "january" and string_2 as "jane". The first two characters from string_1 and string_2 (j and j) are compared. As they are equal, the second two characters are compared (a and a). Because they are also equal, the third two characters (n and n) are compared. Since the third characters are also equal, the fourth character from both the strings are compared and because 'u' has greater ASCII value than 'e', string_1 is greater than string_2. You can even compare a string against an empty string.

5.2.2 Built-In Functions Used on Strings

There are many built-in functions for which a string can be passed as an argument (TABLE 5.1).

TABLE 5.1

Built-In Functions Used on Strings

Built-In Functions	Description
len()	The *len()* function calculates the number of characters in a string. The white space characters are also counted.
max()	The *max()* function returns a character having highest ASCII value.
min()	The *min()* function returns character having lowest ASCII value.

For example,

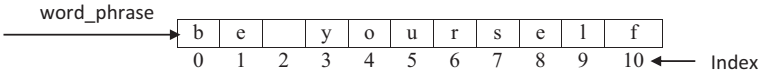
```
1. >>> count_characters = len("eskimos")
2. >>> count_characters
   7
3. >>> max("axel")
   'x'
4. >>> min("brad")
   'a'
```

The number of characters in the string "*eskimos*" ① is calculated using *len()* function ②. Characters with highest and lowest ASCII value are calculated using *max()* ③ and *min()* ④ functions.

5.3 Accessing Characters in String by Index Number

Each character in the string occupies a position in the string. Each of the string's character corresponds to an index number. The first character is at index 0; the next character is at index 1, and so on. The length of a string is the number of characters in it. You can access each character in a string using a subscript operator i.e., a square bracket. Square brackets are used to perform indexing in a string to get the value at a specific index or position. This is also called *subscript* operator.

The index breakdown for the string *"be yourself"* assigned to *word_phrase* string variable is shown below.



The syntax for accessing an individual character in a string is as shown below.

string_name[index]

where *index* is usually in the range of 0 to one less than the length of the string. The value of index should always be an integer and indicates the character to be accessed.

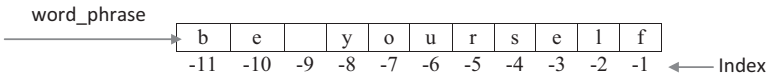
For example,

```
1. >>> word_phrase = "be yourself"
2. >>> word_phrase[0]
   'b'
3. >>> word_phrase[1]
   'e'
4. >>> word_phrase[2]
   ' '
5. >>> word_phrase[3]
   'y'
```

6. >>> word_phrase[10]
   'f'
7. >>> word_phrase[11]
   Traceback (most recent call last):
   File "<stdin>", line 1, in <module>
   IndexError: string index out of range

By referring to the index numbers in square bracket, you can access individual characters in a string. The index number starts with zero corresponding to the first character in the string ②. The index number increases by one as we move to access the next letter to the right of the current letter ③–⑥. The whitespace character between *be* and *yourself* has its own index number, i.e., 2. The last character in the string is referenced by an index value which is the (size of the string – 1) or (len(string) – 1). If you try to specify an index number more than the number of characters in the string, then it results in *IndexError: string index out of range* error ⑦.

You can also access individual characters in a string using negative indexing. If you have a long string and want to access end characters in the string, then you can count backward from the end of the string starting from an index number of –1. The negative index breakdown for the string "*be yourself*" assigned to *word_phrase* string variable is shown below.

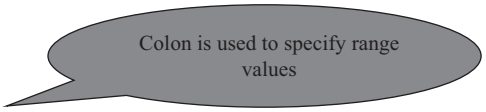


1. >>> word_phrase[-1]
   'f'
2. >>> word_phrase[-2]
   'l'

By using negative index number of –1, you can print the character 'f' ①, the negative index number of –2 prints the character 'l' ②. You can benefit from using negative indexing when you want to access characters at the end of a long string.

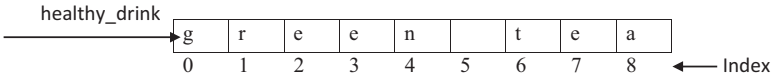
5.4 String Slicing and Joining

The "slice" syntax is a handy way to refer to sub-parts of sequence of characters within an original string. The syntax for string slicing is,



string_name[start:end[:step]]

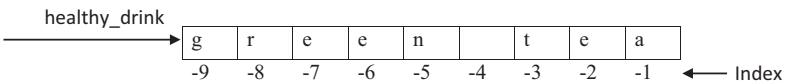
With string slicing, you can access a sequence of characters by specifying a range of index numbers separated by a colon. String slicing returns a sequence of characters beginning at *start* and extending up to but not including *end*. The *start* and *end* indexing values have to be integers. String slicing can be done using either positive or negative indexing.



1. >>> healthy_drink = "green tea"
2. >>> healthy_drink[0:3]
   'gre'
3. >>> healthy_drink[:5]
   'green'
4. >>> healthy_drink[6:]
   'tea'
5. >>> healthy_drink[:]
   'green tea'
6. >>> healthy_drink[4:4]
   ''
7. >>> healthy_drink[6:20]
   'tea'

A substring is created when slicing the strings, which is basically a string that already exists within another string. A substring is any sequence of characters that is contained in a string. The string *"green tea"* is assigned to *healthy_drink* variable ① and a sequence of characters or a substring is extracted from the beginning (0th Index) up to the third character (2nd Index) ②. In string slicing, *start* index value (including) is where slicing starts and *end* index value (excluding) is where the slicing ends. If the *start* index is omitted, the slicing starts from the first index number (0th Index) up to the *end* index (excluding) in the string. In ③ substring starting from 0th index to 4th index is printed. If the *end* index is omitted, slicing starts from the *start* index and goes up to the end of the string index. Substring starting from 6th index to the end of the string is displayed in ④. If both the *start* and *end* index values are omitted then the entire string is displayed ⑤. If the *start* index is equal to or higher than the *end* index, then it results in an empty string ⑥. If the *end* index number is beyond the end of the string, it stops at the end of the string ⑦.

Slicing can also be done using the negative integer numbers.



The negative index can be used to access individual characters in a string. Negative indexing starts with –1 index corresponding to the last character in the string and then the index decreases by one as we move to the left.

```
1. >>> healthy_drink[-3:-1]
   'te'
2. >>> healthy_drink[6:-1]
   'te'
```

You need to specify the lowest negative integer number in the start index position when using negative index numbers as it occurs earlier in the string ①. You can also combine positive and negative indexing numbers ②.

5.4.1 Specifying Steps in Slice Operation

In the slice operation, a third argument called *step* which is an optional can be specified along with the *start* and *end* index numbers. This *step* refers to the number of characters that can be skipped after the *start* indexing character in the string. The default value of *step* is one. In the previous slicing examples, *step* is not specified and in its absence, a default value of one is used. For example,

```
1. >>> newspaper = "new york times"
2. >>> newspaper[0:12:4]
   'ny'
3. >>> newspaper[::4]
   'ny e'
```

In ② the slice [0:12:4] takes the character at 0th index which is *n* and will print every 4th character in the string extending till the 12th character (excluding). You can omit both *start* and *end* index values to consider the entire range of characters in the string by specifying two colons while considering the step argument which will specify the number of characters to skip ③.

Program 5.1: Write Python Code to Determine Whether the Given String Is a Palindrome or Not Using Slicing

```
1. def main():
2.     user_string = input("Enter string: ")
3.     if user_string == user_string[::-1]:
4.         print(f"User entered string is palindrome")
5.     else:
6.         print(f"User entered string is not a palindrome")
7. if __name__ == "__main__":
8.     main()
```

Output

```
Case 1:
Enter string: madam
User entered string is palindrome
```

Case 2:
Enter string: cat
User entered string is not a palindrome

User entered string value is stored in *user_string* variable ②. The *user_string* is reversed using string slicing with –1 ③ and is compared with non-reversed *user_string*. If both are equal, then the string is palindrome ④ else it is not a palindrome ⑥.

5.4.2 Joining Strings Using *join()* Method

Strings can be joined with the *join()* string. The *join()* method provides a flexible way to concatenate strings. The syntax of *join()* method is,

string_name.join(sequence)

Here sequence can be string or list. If the sequence is a string, then *join()* function inserts *string_name* between each character of the string sequence and returns the concatenated string. If the sequence is a list, then *join()* function inserts *string_name* between each item of list sequence and returns the concatenated string. It should be noted that all the items in the list should be of string type.

```
1. >>> date_of_birth = ["17", "09", "1950"]
2. >>> ":".join(date_of_birth)
   '17:09:1950'
3. >>> social_app = ["instagram", "is", "an", "photo", "sharing", "application"]
4. >>> " ".join(social_app)
   'instagram is an photo sharing application'
5. >>> numbers = "123"
6. >>> characters = "amy"
7. >>> password = numbers.join(characters)
8. >>> password
   'a123m123y'
```

All the items in the list *date_of_birth* list variable is of string type ①. In ② the string ":" is inserted between each list item and the concatenated string is displayed. In *social_app* list variable, all the list items are of string type ③. In ④, the *join()* method ensures a blank space is inserted between each of the list items in *social_app* and the concatenated string is displayed. The variables *numbers* ⑤ and *characters* ⑥ are of string type. The string value of "123" is placed between each character of "amy" string resulting in 'a123m123y'. The string value of "123" is inserted between *a* and *m* and again between *m* and *y* and is assigned to *password* string variable.

5.4.3 Split Strings Using *split()* Method

The *split()* method returns a list of string items by breaking up the string using the delimiter string. The syntax of *split()* method is,

string_name.split([separator [, maxsplit]])

Here *separator* is the delimiter string and is optional. A given string is split into list of strings based on the specified *separator*. If the *separator* is not specified then whitespace is considered as the delimiter string to separate the strings. If *maxsplit* is given, at most *maxsplit* splits are done (thus, the list will have at most maxsplit + 1 items). If *maxsplit* is not specified or –1, then there is no limit on the number of splits.

```
1. >>> inventors = "edison, tesla, marconi, newton"
2. >>> inventors.split(",")
   ['edison', ' tesla', ' marconi', ' newton']
3. >>> watches = "rolex hublot cartier omega"
4. >>> watches.split()
   ['rolex', 'hublot', 'cartier', 'omega']
```

The value in *inventors* string variable ① is separated based on "," (comma) ② separator. In ④ no separator is specified in *split()* method. Hence, the string variable *watches* ③ is separated based on whitespace.

5.4.4 Strings Are Immutable

As strings are immutable, it cannot be modified. The characters in a string cannot be changed once a string value is assigned to string variable. However, you can assign different string values to the same string variable.

```
1. >>> immutable = "dollar"
2. >>> immutable[0] = "c"
   Traceback (most recent call last):
     File "<stdin>", line 1, in <module>
   TypeError: 'str' object does not support item assignment
3. >>> string_immutable = "c" + immutable[1:]
4. >>> string_immutable
   'collar'
5. >>> immutable = "rollar"
6. >>> immutable
   'rollar'
```

The string value "*dollar*" is assigned to string variable *immutable* ①. If you try to change the string by assigning a character in place of existing character through indexing, then it results in an error as the string is immutable ②. You cannot change the string value once it has been assigned to a string variable. What you can do is create a new string that is a variation of the original string ③–④. Also, assigning a whole new string value to existing string variable is allowed ⑤–⑥.

5.4.5 String Traversing

Since the string is a sequence of characters, each of these characters can be traversed using the *for* loop.

Example:

```
# Python program to iterate over characters of a string

string_name = "Bangalore"

for element in string_name:

   print(f"Character is {element}")
```

```
Output:
Character is B
Character is a
Character is n
Character is g
Character is a
Character is l
Character is o
Character is r
Character is e
```

5.5 String Methods

You can get a list of all the methods associated with string (TABLE 5.2) by passing the *str* function to dir().

1. >>> dir(str)

['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']

TABLE 5.2

Various String Methods

String Methods	Syntax	Description
capitalize()	string_name.capitalize()	The *capitalize()* method returns a copy of the string with its first character capitalized and the rest lowercased.
casefold()	string_name.casefold()	The *casefold()* method returns a casefolded copy of the string. Casefolded strings may be used for caseless matching.
center()	string_name.center(width[, fillchar])	The method *center()* makes string_name centered by taking width parameter into account. Padding is specified by parameter fillchar. Default filler is a space.
count()	string_name.count(substring [, start [, end]])	The method *count()*, returns the number of non-overlapping occurrences of substring in the range [start, end]. Optional arguments start and end are interpreted as in slice notation.
endswith()	string_name.endswith(suffix[, start[, end]])	This method *endswith()*, returns *True* if the string_name ends with the specified suffix substring, otherwise returns *False*. With optional start, test beginning at that position. With optional end, stop comparing at that position.
find()	string_name. find(substring[, start[, end]])	Checks if substring appears in string_name or if substring appears in string_name specified by starting index *start* and ending index *end*. Return position of the first character of the first instance of string substring in string_name, otherwise return –1 if substring not found in string_name.
isalnum()	string_name.isalnum()	The method *isalnum()* returns Boolean *True* if all characters in the string are alphanumeric and there is at least one character, else it returns Boolean *False*.
isalpha()	string_name.isalpha()	The method *isalpha()*, returns Boolean *True* if all characters in the string are alphabetic and there is at least one character, else it returns Boolean *False*.
isdecimal()	string_name.isdecimal()	The method *isdecimal()*, returns Boolean *True* if all characters in the string are decimal characters and there is at least one character, else it returns Boolean *False*.
isdigit()	string_name.isdigit()	The method *isdigit()* returns Boolean *True* if all characters in the string are digits and there is at least one character, else it returns Boolean *False*.
isidentifier()	string_name.isidentifier()	The method *isidentifier()* returns Boolean *True* if the string is a valid identifier, else it returns Boolean *False*.
islower()	string_name.islower()	The method *islower()* returns Boolean *True* if all characters in the string are lowercase, else it returns Boolean *False*.
isspace()	string_name.isspace()	The method *isspace()* returns Boolean *True* if there are only whitespace characters in the string and there is at least one character, else it returns Boolean *False*.
isnumeric()	string_name.isnumeric()	The method *isnumeric()*, returns Boolean *True* if all characters in the string_name are numeric characters, and there is at least one character, else it returns Boolean *False*. Numeric characters include digit characters and all characters that have the Unicode numeric value property.

(*Continued*)

TABLE 5.2 (*Continued*)

Various String Methods

String Methods	Syntax	Description
istitle()	string_name.istitle()	The method *istitle()* returns Boolean *True* if the string is a title cased string and there is at least one character, else it returns Boolean *False*.
isupper()	string_name.isupper()	The method *isupper()* returns Boolean *True* if all cased characters in the string are uppercase and there is at least one cased character, else it returns Boolean *False*.
upper()	string_name.upper()	The method *upper()* converts lowercase letters in string to uppercase.
lower()	string_name.lower()	The method *lower()* converts uppercase letters in string to lowercase.
ljust()	string_name.ljust(width[, fillchar])	In the method *ljust()*, when you provide the string to the method *ljust()*, it returns the string left justified. Total length of string is defined in first parameter of method width. Padding is done as defined in second parameter fillchar. (default is space).
rjust()	string_name.rjust(width[, fillchar])	In the method *rjust()*, when you provide the string to the method *rjust()*, it returns the string right justified. The total length of string is defined in the first parameter of the method, width. Padding is done as defined in second parameter fillchar. (default is space).
title()	string_name.title()	The method *title()* returns "titlecased" versions of string, that is, all words begin with uppercase characters and the rest are lowercase.
swapcase()	string_name.swapcase()	The method *swapcase()* returns a copy of the string with uppercase characters converted to lowercase and vice versa.
splitlines()	string_name. splitlines([keepends])	The method *splitlines()* returns a list of the lines in the string, breaking at line boundaries. Line breaks are not included in the resulting list unless keepends is given and true.
startswith()	string_name.startswith(prefix[, start[, end]])	The method *startswith()* returns Boolean *True* if the string starts with the prefix, otherwise return *False*. With optional start, test string_name beginning at that position. With optional end, stop comparing string_ name at that position.
strip()	string_name.strip([chars])	The method *lstrip()* returns a copy of the string_name in which specified *chars* have been stripped from both side of the string. If char is not specified then space is taken as default.
rstrip()	string_name.rstrip([chars])	The method *rstrip()* removes all trailing whitespace of string_name.
lstrip()	string_name.lstrip([chars])	The method *lstrip()* removes all leading whitespace in string_name.
replace()	string_name. replace(old, new[, max])	The method *replace()* replaces all occurrences of old in string_name with new. If the optional argument max is given, then only the first max occurrences are replaced.
zfill()	string_name.zfill(width)	The method *zfill()* pads the string_name on the left with zeros to fill width.

Note: Replace the word *string_name* mentioned in the syntax with the actual string name in your code.

For example,

```
1. >>> fact = "Abraham Lincoln was also a champion wrestler"
2. >>> fact.isalnum()
   False
3. >>> "sailors".isalpha()
   True
4. >>> "2018".isdigit()
   True
5. >>> fact.islower()
   False
6. >>> "TSAR BOMBA".isupper()
   True
7. >>> "columbus".islower()
   True
8. >>> warriors = "ancient gladiators were vegetarians"
9. >>> warriors.endswith("vegetarians")
   True
10. >>> warriors.startswith("ancient")
    True
11. >>> warriors.startswith("A")
    False
12. >>> warriors.startswith("a")
    True
13. >>> "cucumber".find("cu")
    0
14. >>> "cucumber".find("um")
    3
15. >>> "cucumber".find("xyz")
    -1
16. >>> warriors.count("a")
    5
17. >>> species = "charles darwin discovered galapagos tortoises"
18. >>> species.capitalize()
    'Charles darwin discovered galapagos tortoises'
19. >>> species.title()
    'Charles Darwin Discovered Galapagos Tortoises'
20. >>> "Tortoises".lower()
    'tortoises'
```

```
21. >>> "galapagos".upper()
    'GALAPAGOS'
22. >>> "Centennial Light".swapcase()
    'cENTENNIAL lIGHT'
23. >>> "history does repeat".replace("does", "will")
    'history will repeat'
24. >>> quote = "   Never Stop Dreaming   "
25. >>> quote.rstrip()
    '   Never Stop Dreaming'
26. >>> quote.lstrip()
    'Never Stop Dreaming   '
27. >>> quote.strip()
    'Never Stop Dreaming'
28. >>> 'ab c\n\nde fg\rkl\r\n'.splitlines()
    ['ab c', '', 'de fg', 'kl']
29. >>> "scandinavian countries are rich".center(40)
    'scandinavian countries are rich'
```

Various operations on strings are carried out using string ①–㉙ methods.

String methods like *capitalize()*, *lower()*, *upper()*, *swapcase()*, *title()* and *count()* are used for conversion purpose. String methods like *islower()*, *isupper()*, *isdecimal()*, *isdigit()*, *isnumeric()*, *isalpha()* and *isalnum()* are used for comparing strings. Some of the string methods used for padding are *rjust()*, *ljust()*, *zfill()* and *center()*. The string method *find()* is used to find substring in an existing string. You can use string methods like *replace()*, *join()*, *split()* and *splitlines()* to replace a string in Python.

Program 5.6: Write Python Program That Accepts a Sentence and Calculate the Number of Words, Digits, Uppercase Letters and Lowercase Letters

```
1. def string_processing(user_string):
2.     word_count = 0
3.     digit_count = 0
4.     upper_case_count = 0
5.     lower_case_count = 0
6.     for each_char in user_string:
7.         if each_char.isdigit():
8.             digit_count += 1
9.         elif each_char.isspace():
10.            word_count += 1
11.        elif each_char.isupper():
12.            upper_case_count += 1
```

6

Lists

AIM

Understand the role of lists in Python in storing multiple items of different types and perform manipulations using various methods.

LEARNING OUTCOMES

At the end of the chapter, you are expected to

- Create and manipulate items in lists.
- Comprehend Indexing and slicing in lists.
- Use methods associated with lists.
- Using lists as arguments in functions.
- Use *for* loop to access individual items in lists.

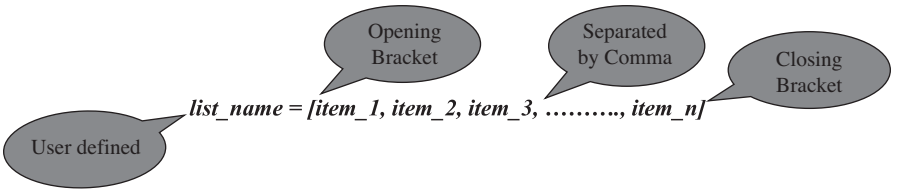
Most of the times a variable can hold a single value. However, in many cases, you need to assign more than that. Consider a Forest scenario where animals belonging to different types of families live. Some of them like Lion, Tiger, Cheetah are Carnivorous and others like Monkeys, Elephants and Buffalos are Herbivorous. If you want to define this habitat computationally, then you have to create multiple variables to represent each animal belonging to a particular animal family which may not make much sense. Instead, you can combine all the animals belonging to the particular animal family under one variable and then use this variable name for further manipulation.

You can think of the list as a container that holds a number of items. Each element or value that is inside a list is called an item. All the items in a list are assigned to a single variable. Lists avoid having a separate variable to store each item which is less efficient and more error prone when you have to perform some operations on these items. Lists can be simple or nested lists with varying types of values. Lists are one of the most flexible data storage formats in Python because they can have values added, removed, and changed.

6.1 Creating Lists

Lists are constructed using square brackets [ ] wherein you can include a list of items separated by commas.

The syntax for creating list is,




```
1. >>> superstore = ["metro", "tesco", "walmart", "kmart", "carrefour"]
2. >>> superstore
   ['metro', 'tesco', 'walmart', 'kmart', 'carrefour']
```

In ① each item in the list is a string. The contents of the list variable are displayed by executing the list variable name ②. When you print out the list, the output looks exactly like the list you had created.

You can create an empty list without any items. The syntax is,

list_name = [ ]

For example,

```
1. >>> number_list = [4, 4, 6, 7, 2, 9, 10, 15]
2. >>> mixed_list = ['dog', 87.23, 65, [9, 1, 8, 1]]
3. >>> type(mixed_list)
   <class 'list'>
4. >>> empty_list = []
5. >>> empty_list
   []
6. >>> type(empty_list)
   <class 'list'>
```

Here, *number_list* ① contains items of the same type while in *mixed_list* ② the items are a mix of type string, float, integer and another list itself. You can determine the type of a *mixed_list* ③ variable by passing the variable name as an argument to *type()* function. In Python, the list type is called as *list*. An empty list can be created as shown in ④–⑤ and the variable *empty_list* is of *list* type ⑥.



You can store any item in a list like string, number, object, another variable and even another list. You can have a mix of different item types and these item types need not have to be homogeneous. For example, you can have a list which is a mix of type numbers, strings and another list itself.

6.2 Basic List Operations

In Python, lists can also be concatenated using the + sign, and the * operator is used to create a repeated sequence of list items. For example,

```
1. >>> list_1 = [1, 3, 5, 7]
2. >>> list_2 = [2, 4, 6, 8]
3. >>> list_1 + list_2
   [1, 3, 5, 7, 2, 4, 6, 8]
4. >>> list_1 * 3
   [1, 3, 5, 7, 1, 3, 5, 7, 1, 3, 5, 7]
5. >>> list_1 == list_2
   False
```

Two lists containing numbers as items are created ①–②. The *list_1* and *list_2* lists are added to form a new list. The new list has all items of both the lists ③. You can use the multiplication operator on the list. It repeats the items the number of times you specify and in ④ the *list_1* contents are repeated three times. Contents of the lists *list_1* and *list_2* are compared using the == operator ⑤ and the result is Boolean *False* since the items in both the lists are different.

You can check for the presence of an item in the list using *in* and *not in* membership operators. It returns a Boolean *True* or *False*. For example,

```
1. >>> list_items = [1,3,5,7]
2. >>> 5 in list_items
   True
3. >>> 10 in list_items
   False
```

If an item is present in the list then using *in* operator results in *True* ② else returns *False* Boolean value ③.

6.2.1 The *list()* Function

The built-in *list()* function is used to create a list. The syntax for *list()* function is,

list([sequence])

where the sequence can be a string, tuple or list itself. If the optional sequence is not specified then an empty list is created. For example,

```
1. >>> quote = "How you doing?"
2. >>> string_to_list = list(quote)
3. >>> string_to_list
   ['H', 'o', 'w', ' ', 'y', 'o', 'u', ' ', 'd', 'o', 'i', 'n', 'g', '?']
4. >>> friends = ["j", "o", "e", "y"]
```

5. >>> friends + quote
   Traceback (most recent call last):
     File "<stdin>", line 1, in <module>
   TypeError: can only concatenate list (not "str") to list
6. >>> friends + list(quote)
   ['j', 'o', 'e', 'y', 'H', 'o', 'w', ' ', 'y', 'o', 'u', ' ', 'd', 'o', 'i', 'n', 'g', '?']

The string variable *quote* ① is converted to a list using the *list()* function ②. Now, let's see whether a string can be concatenated with a list ⑤. The result is an Exception which says *TypeError: can only concatenate list (not "str") to list*. That means you cannot concatenate a list with a string value. In order to concatenate a string with the list, you must convert the string value to list type, using Python built-in *list()* function ⑥.

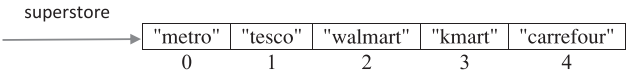
6.3 Indexing and Slicing in Lists

As an ordered sequence of elements, each item in a list can be called individually, through indexing. The expression inside the bracket is called the index. Lists use square brackets [ ] to access individual items, with the first item at index 0, the second item at index 1 and so on. The index provided within the square brackets indicates the value being accessed.

The syntax for accessing an item in a list is,

list_name[index]

where index should always be an integer value and indicates the item to be selected. For the list *superstore*, the index breakdown is shown below.

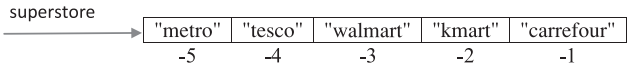


1. >>> superstore = ["metro", "tesco", "walmart", "kmart", "carrefour"]
2. >>> superstore[0]
   'metro'
3. >>> superstore[1]
   'tesco'
4. >>> superstore[2]
   'walmart'
5. >>> superstore[3]
   'kmart'
6. >>> superstore[4]
   'carrefour'
7. >>> superstore[9]
   Traceback (most recent call last):
     File "<stdin>", line 1, in <module>
   IndexError: list index out of range

The *superstore* list has five items. To print the first item in the list use square brackets immediately after list name with an index value of zero ②. The index numbers for this *superstore* list range from 0 to 4 ②–⑥. If the index value is more than the number of items in the list ⑦ then it results in *"IndexError: list index out of range"* error.

In addition to positive index numbers, you can also access items from the list with a negative index number, by counting backwards from the end of the list, starting at –1. Negative indexing is useful if you have a long list and you want to locate an item towards the end of a list.

For the same list *superstore*, the negative index breakdown is shown below.




```
1. >>> superstore[-3]
   'walmart'
```

If you would like to print out the item 'walmart' by using its negative index number, you can do so as in ①.

6.3.1 Modifying Items in Lists

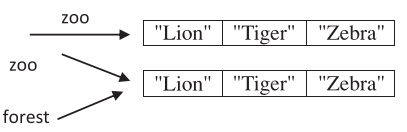
Lists are mutable in nature as the list items can be modified after you have created a list. You can modify a list by replacing the older item with a newer item in its place and without assigning the list to a completely new variable. For example,

```
1. >>> fauna = ["pronghorn", "alligator", "bison"]
2. >>> fauna[0] = "groundhog"
3. >>> fauna
   ['groundhog', 'alligator', 'bison']
4. >>> fauna[2] = "skunk"
5. >>> fauna
   ['groundhog', 'alligator', 'skunk']
6. >>> fauna[-1] = "beaver"
7. >>> fauna
   ['Groundhog', 'alligator', 'beaver']
```

You can change the string item at index 0 from 'pronghorn' to 'groundhog' as shown in ②. Now when you display *fauna,* the list items will be different ③. The item at index 2 is changed from "bison" to "skunk" ④. You can also change the value of an item by using a negative index number –1 ⑥ which corresponds to the positive index number of 2. Now "skunk" is replaced with "beaver" ⑦.

When you assign an existing list variable to a new variable, an assignment (=) on lists does not make a new copy. Instead, assignment makes both the variable names point to the same list in memory. For example,

```
1. >>> zoo = ["Lion", "Tiger", "Zebra"]
2. >>> forest = zoo
3. >>> type(zoo)
```

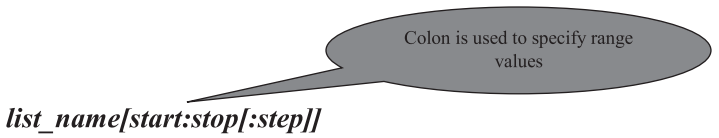


```
    <class 'list'>
4. >>> type(forest)
    <class 'list'>
5. >>> forest
    ['Lion', 'Tiger', 'Zebra']
6. >>> zoo[0] = "Fox"
7. >>> zoo
    ['Fox', 'Tiger', 'Zebra']
8. >>> forest
    ['Fox', 'Tiger', 'Zebra']
9. >>> forest[1] = "Deer"
10. >>> forest
    ['Fox', 'Deer', 'Zebra']
11. >>> zoo
    ['Fox', 'Deer', 'Zebra']
```

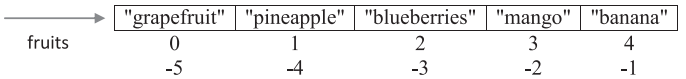
The above code proves that assigning an existing list variable to a new variable does not create a new copy of the existing list items ①–⑪.

Slicing of lists is allowed in Python wherein a part of the list can be extracted by specifying index range along with the colon (:) operator which itself is a list.

The syntax for list slicing is,



where both *start* and *stop* are integer values (positive or negative values). List slicing returns a part of the list from the *start* index value to *stop* index value which includes the *start* index value but excludes the *stop* index value. *Step* specifies the increment value to slice by and it is optional. For the list *fruits*, the positive and negative index breakdown is shown below.




```
1. >>> fruits = ["grapefruit", "pineapple", "blueberries", "mango", "banana"]
2. >>> fruits[1:3]
    ['pineapple', 'blueberries']
3. >>> fruits[:3]
    ['grapefruit', 'pineapple', 'blueberries']
4. >>> fruits[2:]
    ['blueberries', 'mango', 'banana']
5. >>> fruits[1:4:2]
    ['pineapple', 'mango']
```

6. >>> fruits[:]
   ['grapefruit', 'pineapple', 'blueberries', 'mango', 'banana']
7. >>> fruits[::2]
   ['grapefruit', 'blueberries', 'banana']
8. >>> fruits[::-1]
   ['banana', 'mango', 'blueberries', 'pineapple', 'grapefruit']
9. >>> fruits[-3:-1]
   ['blueberries', 'mango']

All the items in the *fruits* list starting from an index value of 1 up to index value of 3 but excluding the index value of 3 is sliced ②. If you want to access the *start* items, then there is no need to specify the index value of zero. You can skip the *start* index value and specify only the *stop* index value ③. Similarly, if you want to access the last *stop* items, then there is no need to specify the *stop* value and you have to mention only the *start* index value ④. When *stop* index value is skipped the range of items accessed extends up to the last item. If you skip both the *start* and *stop* index values ⑥ and specify only the colon operator within the brackets, then the entire items in the list are displayed. The number after the second colon tells Python that you would like to choose your slicing increment. By default, Python sets this increment to 1, but that number after second colon allows you to specify what you want it to be. Using double colon as shown in ⑦ means no value for *start* index and no value for *stop* index and jump the items by two steps. Every second item of the list is extracted starting from an index value of zero. All the items in a list can be displayed in reverse order by specifying a double colon followed by an index value of –1 ⑧. Negative index values can also be used for *start* and *stop* index values ⑨.

6.4 Built-In Functions Used on Lists

There are many built-in functions for which a list can be passed as an argument (TABLE 6.1).

TABLE 6.1

Built-In Functions Used on Lists

Built-In Functions	Description
len()	The *len()* function returns the numbers of items in a list.
sum()	The *sum()* function returns the sum of numbers in the list.
any()	The *any()* function returns *True* if any of the Boolean values in the list is *True*.
all()	The *all()* function returns *True* if all the Boolean values in the list are *True*, else returns *False*.
sorted()	The *sorted()* function returns a modified copy of the list while leaving the original list untouched.

For example,

1. >>> lakes = ['superior', 'erie', 'huron', 'ontario', 'powell']
2. >>> len(lakes)
   5

```
3. >>> numbers = [1, 2, 3, 4, 5]
4. >>> sum(numbers)
   15
5. >>> max(numbers)
   5
6. >>> min(numbers)
   1
7. >>> any([1, 1, 0, 0, 1, 0])
   True
8. >>> all([1, 1, 1, 1])
   True
9. >>> lakes_sorted_new = sorted(lakes)
10. >>> lakes_sorted_new
    ['erie', 'huron', 'ontario', 'powell', 'superior']
```

You can find the number of items in the list *lakes* using the *len()* function ②. Using the *sum()* function results in adding up all the numbers in the list ④. Maximum and minimum numbers in a list are returned using *max()* ⑤ and *min()* functions ⑥. If any of the items is 1 in the list then *any()* function returns Boolean *True* value ⑦. If all the items in the list are 1 then *all()* function returns *True* else returns *False* Boolean value ⑧. The *sorted()* function returns the sorted list of items without modifying the original list which is assigned to a new list variable ⑨. In the case of string items in the list, they are sorted based on their ASCII values.

6.5 List Methods

The list size changes dynamically whenever you add or remove the items and there is no need for you to manage it yourself. You can get a list of all the methods (TABLE 6.2) associated with the *list* by passing the list function to *dir()*.

```
1. >>> dir(list)
   ['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
   '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__',
   '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__',
   '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
   '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__sub-
   classhook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove',
   'reverse', 'sort']
```

Various methods associated with *list* is displayed ①.

TABLE 6.2

Various List Methods

List Methods	Syntax	Description
append()	list.append(item)	The *append()* method adds a single item to the end of the list. This method does not return new list and it just modifies the original.
count()	list.count(item)	The *count()* method counts the number of times the item has occurred in the list and returns it.
insert()	list.insert(index, item)	The *insert()* method inserts the item at the given index, shifting items to the right.
extend()	list.extend(list2)	The *extend()* method adds the items in list2 to the end of the list.
index()	list.index(item)	The *index()* method searches for the given item from the start of the list and returns its index. If the value appears more than once, you will get the index of the first one. If the item is not present in the list then *ValueError* is thrown by this method.
remove()	list.remove(item)	The *remove()* method searches for the first instance of the given item in the list and removes it. If the item is not present in the list then *ValueError* is thrown by this method.
sort()	list.sort()	The *sort()* method sorts the items *in place* in the list. This method modifies the original list and it does not return a new list.
reverse()	list.reverse()	The *reverse()* method reverses the items *in place* in the list. This method modifies the original list and it does not return a new list.
pop()	list.pop([index])	The *pop()* method removes and returns the item at the given index. This method returns the rightmost item if the index is omitted.

Note: Replace the word *"list"* mentioned in the syntax with your *actual list name* in your code.

For example,

```
1. >>> cities = ["oslo", "delhi", "washington", "london", "seattle", "paris", "washington"]
2. >>> cities.count('seattle')
   1
3. >>> cities.index('washington')
   2
4. >>> cities.reverse()
5. >>> cities
   ['washington', 'paris', 'seattle', 'london', 'washington', 'delhi', 'oslo']
6. >>> cities.append('brussels')
7. >>> cities
   ['washington', 'paris', 'seattle', 'london', 'washington', 'delhi', 'oslo', 'brussels']
8. >>> cities.sort()
9. >>> cities
   ['brussels', 'delhi', 'london', 'oslo', 'paris', 'seattle', 'washington', 'washington']
10. >>> cities.pop()
    'washington'
```

```
11. >>> cities
    ['brussels', 'delhi', 'london', 'oslo', 'paris', 'seattle', 'washington']
12. >>> more_cities = ["brussels", "copenhagen"]
13. >>> cities.extend(more_cities)
14. >>> cities
    ['brussels', 'delhi', 'london', 'oslo', 'paris', 'seattle', 'washington', 'brussels', 'copenhagen']
15. >>> cities.remove("brussels")
16. >>> cities
    ['delhi', 'london', 'oslo', 'paris', 'seattle', 'washington', 'brussels', 'copenhagen']
```

Various operations on lists are carried out using list methods ①–⑯.

6.5.1 Populating Lists with Items

One of the popular way of populating lists is to start with an empty list [ ], then use the functions *append()* or *extend()* to add items to the list. For example,

```
1. >>> continents = []
2. >>> continents.append("Asia")
3. >>> continents.append("Europe")
4. >>> continents.append("Africa")
5. >>> continents
   ['Asia', 'Europe', 'Africa']
```

Create an empty list *continents* ① and start populating items to the *continents* list using append() function ②–④ and finally display the items of the *continents* list ⑤.

Program 6.1: Program to Dynamically Build User Input as a List

```
1. list_items = input("Enter list items separated by a space ").split()
2. print(f"List items are {list_items}")
3. items_of_list = []
4. total_items = int(input("Enter the number of items "))
5. for i in range(total_items):
6.     item = input("Enter list item: ")
7.     items_of_list.append(item)
8. print(f"List items are {items_of_list}")
```

Output

```
Enter list items separated by a space Asia Europe Africa
List items are ['Asia', 'Europe', 'Africa']
```

```
Enter the number of items 2
Enter list item: Australia
Enter list item: Americas
List items are ['Australia', 'Americas']
```

You can build a list from user-entered values in two ways. In the first method ①–② you chain *input()* and *split()* together using dot notation. The user has to enter the items separated by spaces. Even though the user entered input has multiple items separated by spaces, by default the user-entered input is considered a single string. You benefit from the fact that the *split()* method can be used with the string. This user-entered input string value is split based on spaces and the *split()* method returns a list of string items. The advantage of this method is, there is no need for you to specify the total number of items that you are planning to insert into the list. In the second method ③–⑧ you have to specify the total number of items that you are planning to insert into the list beforehand itself ④. Based on this number we iterate through the *for* loop as many times using *range()* function ⑤. During each iteration, you need to append ⑦ the user entered value to the list ⑥.

6.5.2 Traversing of Lists

Using a *for* loop you can iterate through each item in a list.

Program 6.2: Program to Illustrate Traversing of Lists Using the *for* loop

```
1. fast_food = ["waffles", "sandwich", "burger", "fries"]
2. for each_food_item in fast_food:
3.     print(f"I like to eat {each_food_item}")
4. for each_food_item in ["waffles", "sandwich", "burger", "fries"]:
5.     print(f"I like to eat {each_food_item}")
```

Output

```
I like to eat waffles
I like to eat sandwich
I like to eat burger
I like to eat fries
I like to eat waffles
I like to eat sandwich
I like to eat burger
I like to eat fries
```

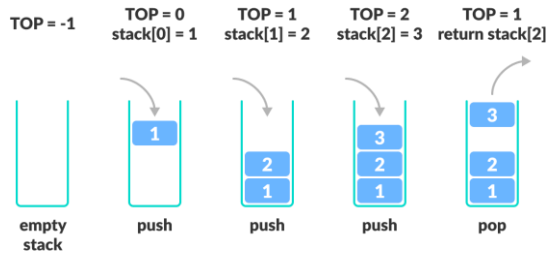
The *for* statement makes it easy to loop over the items in a list. A list variable is created ① and the list variable is specified in the *for* loop ②. Instead of specifying a list variable, you can specify a list directly in the *for* loop ④.

You can obtain the index value of each item in the list by using *range()* along with *len()* function.

Implementation of Stack using List

A stack is a linear data structure that stores items in a Last-In/First-Out (LIFO) or First-In/Last-Out (FILO) manner. In stack, a new element is added at one end and an element is removed from that end only. The insert and delete operations are often called push and pop.

Working of Stack




```
# Stack implementation in python
# Creating a stack
def create_stack():
    stack = []
    return stack

# Creating an empty stack
def check_empty(stack):
    return len(stack) == 0

# Adding items into the stack
def push(stack, item):
    stack.append(item)
    print("pushed item: " + item)

# Removing an element from the stack
def pop(stack):
    if (check_empty(stack)):
        return "stack is empty"

    return stack.pop()

stack = create_stack()
push(stack, str(1))
push(stack, str(2))
push(stack, str(3))
push(stack, str(4))
print("popped item: " + pop(stack))
print("stack after popping an element: " + str(stack))
```

Output:

```
pushed item: 1
pushed item: 2
pushed item: 3
pushed item: 4
popped item: 4
stack after popping an element: ['1', '2', '3']
```

SBC Karkala

Implementation of Queue using List

Queue is a linear data structure that stores items in First In First Out (FIFO) manner. With a queue the least recently added item is removed first.

Queue in Python can be implemented using deque class from the collections module. Deque is preferred over list in the cases where we need quicker append and pop operations from both the ends of container.

```
# Python program to demonstrate queue implementation
# using collections.dequeue

from collections import deque
# Initializing a queue
q = deque()

# Adding elements to a queue
q.append('a')
q.append('b')
q.append('c')

print("Initial queue")
print(q)

# Removing elements from a queue
print("\nElements dequeued from the queue")
print(q.popleft())
print(q.popleft())

print("\nQueue after removing elements")
print(q)
```

Output:

```
Initial queue
deque(['a', 'b', 'c'])

Elements dequeued from the queue
a
b

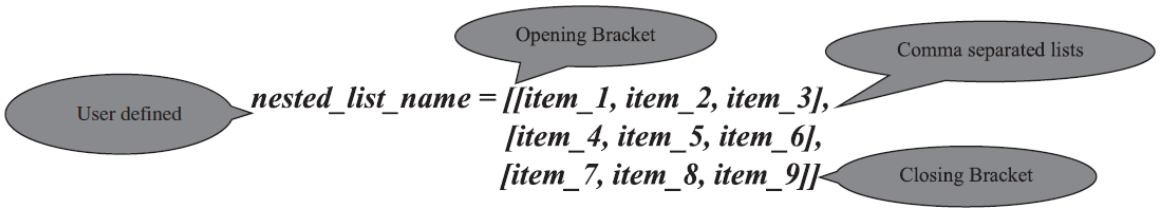
Queue after removing elements
deque(['c'])
```

SBC Karkala

6.5.3 Nested Lists

A list inside another list is called a nested list and you can get the behavior of nested lists in Python by storing lists within the elements of another list. You can traverse through the items of nested lists using the *for* loop.

The syntax for nested lists is,



Each list inside another list is separated by a comma. For example,

```
1. >>> asia = [["India", "Japan", "Korea"],
                ["Srilanka", "Myanmar", "Thailand"],
                ["Cambodia", "Vietnam", "Israel"]]
2. >>> asia[0]
   ['India', 'Japan', 'Korea']
3. >>> asia[0][1]
   'Japan'
4. >>> asia[1][2] = "Philippines"
5. >>> asia
   [['India', 'Japan', 'Korea'], ['Srilanka', 'Myanmar', 'Philippines'], ['Cambodia', 'Vietnam',
   'Israel']]
```

Chapter 3- Dictionaries

A dictionary is a collection of an unordered set of **key:value pairs**, with the requirement that the keys are unique within a dictionary. Dictionaries are constructed using curly braces { }, wherein you include a list of key:value pairs separated by commas. Dictionary keys are case sensitive.

Syntax:

dictionary_name = {key_1:value_1, key_2:value_2, key_3:value_3, …,key_n:value_n}

For example,

```
>>> fish = {"g": "goldfish", "s":"shark", "n": "needlefish", "b":"barramundi"}
>>> fish
{'g': 'goldfish', 's': 'shark', 'n': 'needlefish', 'b': 'barramundi'}
>>> type(fish)
<class 'dict'>
```

We can create an empty dictionary by specifying a pair of curly braces and without any key:value pairs.
The syntax is, **dictionary_name = { }**
For example,

```
>>> empty_dictionary = {}
>>> empty_dictionary
{}
```

In dictionaries, the order of key:value pairs does not matter. For example,

```
>>> pizza = {"pepperoni":3, "calzone":5, "margherita":4}
>>> fav_pizza = {"margherita":4, "pepperoni":3, "calzone":5}
>>> pizza == fav_pizza
True
```

Accessing and Modifying key:value Pairs in Dictionaries

Each individual key:value pair in a dictionary can be accessed through keys by specifying it inside square brackets. The key provided within the square brackets indicates the key:value pair being accessed.

The syntax for accessing the value for a key in the dictionary is, **dictionary_name[key]**

The syntax for modifying the value of an existing key or for adding a new key:value pair to a dictionary is, **dictionary_name[key] = value**

If the key is already present in the dictionary, then the key gets updated with the new value. If the key is not present then the new key:value pair gets added to the dictionary.

SBC Karkala

For example,

renaissance

Keys	"giotto"	"Donatello"	"Michelangelo"	"Botticelli"	"clouet"
Values	1305	1440	1511	1480	1520

```
1. >>> renaissance = {"giotto":1305, "donatello":1440, "michelangelo":1511,
   "botticelli":1480, "clouet":1520}
2. >>> renaissance["giotto"] = 1310
3. >>> renaissance
   {'giotto': 1310, 'donatello': 1440, 'michelangelo': 1511, 'botticelli': 1480, 'clouet': 1520}
4. >>> renaissance["michelangelo"]
   1511
5. >>> renaissance["leonardo"] = 1503
6. >>> renaissance
   {'giotto': 1310, 'donatello': 1440, 'michelangelo': 1511, 'botticelli': 1480, 'clouet': 1520,
   'leonardo': 1503}
7. >>> renaissance["piero"]
   Traceback (most recent call last):
     File "<stdin>", line 1, in <module>
   KeyError: 'piero'
```

You can check for the presence of a key in the dictionary using *in* and *not in* membership operators. It returns either a Boolean *True* or *False* value. For example,

```
1. >>> clothes = {"rainy":"raincoats", "summer":"tees", "winter":"sweaters"}
2. >>> "spring" in clothes
   False
3. >>> "spring" not in clothes
   True
```

7.2.1 The *dict()* Function

The built-in *dict()* function is used to create dictionary. The syntax for *dict()* function when the optional keyword arguments used is,

dict([**kwarg])

The function *dict()* returns a new dictionary initialized from an optional keyword arguments and a possibly empty set of keyword arguments. If no keyword argument is given, an empty dictionary is created. If keyword arguments are given, the keyword arguments and their values of the form *kwarg = value* are added to the dictionary as *key:value* pairs. For example,

```
1. >>> numbers = dict(one=1, two=2, three=3)
2. >>> numbers
   {'one': 1, 'two': 2, 'three': 3}
```

Keyword arguments of the form *kwarg* = value are converted to *key:value* pairs for *numbers* dictionary ①–②.

The syntax for *dict()* function when iterables used is,

dict(iterable[, **kwarg])

You can specify an iterable containing exactly two objects as tuple, the key and value in the *dict()* function. For example

```
1. >>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
   {'sape': 4139, 'jack': 4098, 'guido': 4127}
```

The *dict()* function builds dictionaries directly from sequences of key, value tuple pairs ①.

7.3 Built-In Functions Used on Dictionaries

There are many built-in functions for which a dictionary can be passed as an argument (TABLE 7.1). The main operations on a dictionary are storing a value with some key and extracting the value for a given key.

Built-in Functions	Description
len()	The *len()* function returns the number of items (*key:value* pairs) in a dictionary.
all()	The *all()* function returns Boolean True value if all the keys in the dictionary are True else returns False.
any()	The *any()* function returns Boolean True value if any of the key in the dictionary is True else returns False.
sorted()	The *sorted()* function by default returns a list of items, which are sorted based on dictionary keys.

For example,

```
1. >>> presidents = {"washington":1732, "jefferson":1751, "lincoln":1809,
   "roosevelt":1858, "eisenhower":1890}
2. >>> len(presidents)
   5
3. >>> all_dict_func = {0:True, 2:False}
4. >>> all(all_dict_func)
   False
5. >>> all_dict_func = {1:True, 2:False}
6. >>> all(all_dict_func)
   True
7. >>> any_dict_func = {1:True, 2:False}
8. >>> any(any_dict_func)
   True
9. >>> sorted(presidents)
   ['eisenhower', 'jefferson', 'lincoln', 'roosevelt', 'washington']
10. >>> sorted(presidents, reverse = True)
    ['washington', 'roosevelt', 'lincoln', 'jefferson', 'eisenhower']
11. >>> sorted(presidents.values())
    [1732, 1751, 1809, 1858, 1890]
12. >>> sorted(presidents.items())
    [('eisenhower', 1890), ('jefferson', 1751), ('lincoln', 1809), ('roosevelt', 1858),
    ('washington', 1732)]
```

7.4 Dictionary Methods

Dictionary allows you to store data in *key:value* format without depending on indexing. You can get a list of all the methods associated with *dict* (TABLE 7.2) by passing the *dict* function to *dir()*.

1. >>> dir(dict)

   ['__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']

Various Dictionary Methods

Dictionary Methods	Syntax	Description
clear()	dictionary_name. clear()	The *clear()* method removes all the *key:value* pairs from the dictionary.
fromkeys()	dictionary_name. fromkeys(seq [, value])	The *fromkeys()* method creates a new dictionary from the given sequence of elements with a value provided by the user.
get()	dictionary_name. get(key [, default])	The *get()* method returns the value associated with the specified key in the dictionary. If the key is not present then it returns the default value. If default is not given, it defaults to *None*, so that this method never raises a *KeyError*.
items()	dictionary_name. items()	The *items()* method returns a new view of dictionary's key and value pairs as tuples.
keys()	dictionary_name. keys()	The *keys()* method returns a new view consisting of all the keys in the dictionary.
pop()	dictionary_name. pop(key[, default])	The *pop()* method removes the key from the dictionary and returns its value. If the key is not present, then it returns the default value. If default is not given and the key is not in the dictionary, then it results in *KeyError*.
popitem()	dictionary_name. popitem()	The *popitem()* method removes and returns an arbitrary (key, value) tuple pair from the dictionary. If the dictionary is empty, then calling *popitem()* results in *KeyError*.
setdefault()	dictionary_name. setdefault (key[, default])	The *setdefault()* method returns a value for the key present in the dictionary. If the key is not present, then insert the key into the dictionary with a default value and return the default value. If key is present, *default* defaults to None, so that this method never raises a *KeyError*.
update()	dictionary_name. update([other])	The *update()* method updates the dictionary with the *key:value* pairs from *other* dictionary object and it returns *None*.
values()	dictionary_name. values()	The *values()* method returns a new view consisting of all the values in the dictionary.

Note: Replace the word "*dictionary_name*" mentioned in the syntax with your *actual dictionary name* in your code.

For example,

1. >>> box_office_billion = {"avatar":2009, "titanic":1997, "starwars":2015, "harry-potter":2011, "avengers":2012}
2. >>> box_office_billion_fromkeys=box_office_billion.fromkeys(box_office_billion)
3. >>> box_office_billion_fromkeys

   {'avatar': None, 'titanic': None, 'starwars': None, 'harrypotter': None, 'avengers': None}
4. >>> box_office_billion_fromkeys = box_office_billion.fromkeys(box_office_billion, "billion_dollar")

```
5. >>> box_office_billion_fromkeys
   {'avatar': 'billion_dollar', 'titanic': 'billion_dollar', 'starwars': 'billion_dollar', 'harry-
   potter': 'billion_dollar', 'avengers': 'billion_dollar'}
6. >>> print(box_office_billion.get("frozen"))
   None
7. >>> box_office_billion.get("frozen",2013)
   2013
8. >>> box_office_billion.keys()
   dict_keys(['avatar', 'titanic', 'starwars', 'harrypotter', 'avengers'])
9. >>> box_office_billion.values()
   dict_values([2009, 1997, 2015, 2011, 2012])
10. >>> box_office_billion.items()
    dict_items([('avatar', 2009), ('titanic', 1997), ('starwars', 2015), ('harrypotter', 2011),
    ('avengers', 2012)])
11. >>> box_office_billion.update({"frozen":2013})
12. >>> box_office_billion
    {'avatar': 2009, 'titanic': 1997, 'starwars': 2015, 'harrypotter': 2011, 'avengers': 2012, 'frozen': 2013}
13. >>> box_office_billion.setdefault("minions")
14. >>> box_office_billion
    {'avatar': 2009, 'titanic': 1997, 'starwars': 2015, 'harrypotter': 2011, 'avengers': 2012,
    'frozen': 2013, 'minions': None}
15. >>> box_office_billion.setdefault("ironman", 2013)
16. >>> box_office_billion
    {'avatar': 2009, 'titanic': 1997, 'starwars': 2015, 'harrypotter': 2011, 'avengers': 2012,
    'minions': None, 'ironman': 2013}
17. >>> box_office_billion.pop("avatar")
    2009
18. >>> box_office_billion.popitem()
    ('ironman', 2013)
19. >>> box_office_billion.clear()
    {}
```

The *dict_keys*, *dict_values*, and *dict_items* data types returned by various methods of dictionary are read-only views and cannot be edited directly. If you want to convert *dict_keys*, *dict_values*, and *dict_items* data types returned from *keys()*, *values()*, and *items()* methods to a true list then pass their list, such as returned values, to *list()* function. Whenever there is a modification to the dictionary, it gets reflected in the *views* of these data types. For example,

```
1. >>> list(box_office_billion.keys())
   ['avatar', 'titanic', 'starwars', 'harrypotter', 'avengers']
2. >>> list(box_office_billion.values())
   [2009, 1997, 2015, 2011, 2012]
3. >>> list(box_office_billion.items())
   [('avatar', 2009), ('titanic', 1997), ('starwars', 2015), ('harrypotter', 2011), ('avengers', 2012)]
```

The *dict_keys*, *dict_values*, and *dict_items* returned from *keys()*, *values()*, and *items()* are converted to true list using the *list()* function ①–③.

7.4.1 Populating Dictionaries with *key:value* Pairs

One of the common ways of populating dictionaries is to start with an empty dictionary { }, then use the *update()* method to assign a value to the key using assignment operator. If the key does not exist, then the *key:value* pairs will be created automatically and added to the dictionary.

For example,

```
1. >>> countries = {}
2. >>> countries.update({"Asia":"India"})
3. >>> countries.update({"Europe":"Germany"})
4. >>> countries.update({"Africa":"Sudan"})
5. >>> countries
   {'Asia': 'India', 'Europe': 'Germany', 'Africa': 'Sudan'}
```

7.4.2 Traversing of Dictionary

A *for* loop can be used to iterate over keys or values or *key:value* pairs in dictionaries. If you iterate over a dictionary using a *for* loop, then, by default, you will iterate over the keys. If you want to iterate over the values, use *values()* method and for iterating over the *key:value* pairs, specify the dictionary's *items()* method explicitly. The *dict_keys*, *dict_values*, and *dict_items* data types returned by dictionary methods can be used in *for* loops to iterate over the keys or values or *key:value* pairs.

Access key without using a key()

```
# Python3 code to iterate through all keys in a dictionary

statesAndCapitals = {
        'Gujarat': 'Gandhinagar',
        'Maharashtra': 'Mumbai',
        'Rajasthan': 'Jaipur',
        'Bihar': 'Patna'
}

print('List Of given states:\n')

# Iterating over keys
for state in statesAndCapitals:
        print(state)
```

Output:
List Of given states:

Gujarat
Maharashtra
Rajasthan
Bihar

Iterate through all values using .values()

```
# Python3 code to iterate through all values in a dictionary

statesAndCapitals = {
        'Gujarat': 'Gandhinagar',
        'Maharashtra': 'Mumbai',
        'Rajasthan': 'Jaipur',
        'Bihar': 'Patna'
}

print('List Of given capitals:\n')

# Iterating over values
for capital in statesAndCapitals.values():
        print(capital)
```

Output:
List Of given capitals:
Gandhinagar
Mumbai
Jaipur
Patna

SBC Karkala

Iterate through all key, and value pairs using items()

In this example, we are printing all the key and value pairs present in a dictionary using a items() method.

```
# Python3 code to iterate through all key, value pairs in a dictionary

statesAndCapitals = {
        'Gujarat': 'Gandhinagar',
        'Maharashtra': 'Mumbai',
        'Rajasthan': 'Jaipur',
        'Bihar': 'Patna'
}

print('List Of given states and their capitals:\n')

# Iterating over values
for state, capital in statesAndCapitals.items():
        print(state, ":", capital)
```

Output:

```
List Of given states and their capitals:
Gujarat : Gandhinagar
Maharashtra : Mumbai
Rajasthan : Jaipur
Bihar : Patna
```

SBC Karkala

Chapter 4- Tuples

Creating Tuples

Tuples are used to store multiple items in a single variable. Tuples are immutable. Once a tuple is created, you cannot change its values. A tuple is defined by putting a comma-separated list of values inside parentheses ( ).

Syntax: tuple_name = (item_1, item_2, item_3, ……., item_n)

For example,

```
>>> internet = ("cern", "timbernerslee", "www", 1980)
>>> internet
('cern', 'timbernerslee,' 'www', 1980)
>>> type(internet)
<class 'tuple'>
```

We can create an empty tuple without any values. The syntax is, **tuple_name = ()**
For example,

```
>>> empty_tuple = ()
>>> empty_tuple
()
>>> type(empty_tuple)
<class ‘tuple’>
```

Empty tuples are constructed by an empty pair of parentheses. A tuple with one item is constructed by having a value followed by a comma. It is not sufficient to enclose a single value in parentheses.

Example:

```
>>> singleton = 'hello',
>>> singleton
('hello',)
```

Basic Tuple Operations

We can use the + operator to concatenate tuples together and the * operator to repeat a sequence of tuple items.

For example,

```
1. >>> tuple_1 = (2, 0, 1, 4)
2. >>> tuple_2 = (2, 0, 1, 9)
3. >>> tuple_1 + tuple_2
   (2, 0, 1, 4, 2, 0, 1, 9)
4. >>> tuple_1 * 3
   (2, 0, 1, 4, 2, 0, 1, 4, 2, 0, 1, 4)
5. >>> tuple_1 == tuple_2
   False
```

SBC Karkala

You can check for the presence of an item in a tuple using *in* and *not in* membership operators. It returns a Boolean *True* or *False*. For example,

```
1. >>> tuple_items = (1, 9, 8, 8)
2. >>> 1 in tuple_items
   True
3. >>> 25 in tuple_items
   False
```

The *in* operator returns Boolean *True* if an item is present in the tuple ②, or else returns *False* Boolean value ③.

Comparison operators like <, <=, >, >=, == and != are used to compare tuples. For example,

```
1. >>> tuple_1 = (9, 8, 7)
2. >>> tuple_2 = (9, 1, 1)
3. >>> tuple_1 > tuple_2
   True
4. >>> tuple_1 != tuple_2
   True
```

Tuples are compared position by position. The first item of the first tuple is compared to the first item of the second tuple; if they are not equal then this will be the result of the comparison, or else the second item of the first tuple is compared to the second item of the second tuple; if they are not equal then this will be the result of the comparison, else the third item is considered, and so on ①–④.

8.2.1 The *tuple()* Function

The built-in *tuple()* function is used to create a tuple. The syntax for the *tuple()* function is,

tuple([sequence])

where the sequence can be a number, string or tuple itself. If the optional sequence is not specified, then an empty tuple is created.

```
1. >>> norse = "vikings"
2. >>> string_to_tuple = tuple(norse)
3. >>> string_to_tuple
   ('v', 'i', 'k', 'i', 'n', 'g', 's')
4. >>> zeus = ["g", "o", "d", "o", "f", "s", "k", "y"]
5. >>> list_to_tuple = tuple(zeus)
6. >>> list_to_tuple
   ('g', 'o', 'd', 'o', 'f', 's', 'k', 'y')
```

Indexing and Slicing in Tuples

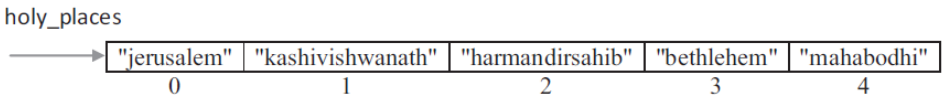
Each item in a tuple can be called individually through indexing. The expression inside the bracket is called the index. Square brackets [ ] are used by tuples to access individual items, with the first item at index 0, the second item at index 1 and so on. The index provided within the square brackets indicates the value being accessed.

The syntax for accessing an item in a tuple is,

tuple_name[index]

where index should always be an integer value and indicates the item to be selected.

For the tuple *holy_places*, the index breakdown is shown below.

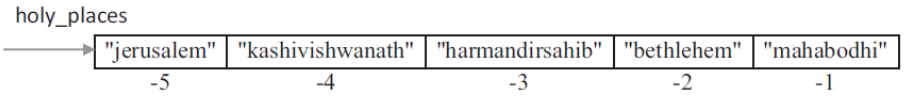


For example,

1. >>> holy_places = ("jerusalem", "kashivishwanath", "harmandirsahib", "bethlehem", "mahabodhi")
2. >>> holy_places
   ('jerusalem', 'kashivishwanath', 'harmandirsahib', 'bethlehem', 'mahabodhi')
3. >>> holy_places[0]
   'jerusalem'
4. >>> holy_places[1]
   'kashivishwanath'

In addition to positive index numbers, you can also access tuple items using a negative index number, by counting backwards from the end of the tuple, starting at –1. Negative indexing is useful if you have a large number of items in the tuple and you want to locate an item towards the end of a tuple.

For the same tuple *holy_places*, the negative index breakdown is shown below.



For example,

1. >>> holy_places[-2]
   'bethlehem'

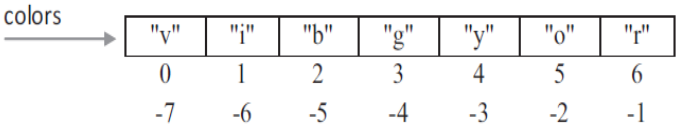
Slicing of tuples is allowed in Python wherein a part of the tuple can be extracted by specifying an index range along with the colon (:) operator, which itself results as tuple type.

The syntax for tuple slicing is,

tuple_name[start:stop[:step]]

where both *start* and *stop* are integer values (positive or negative values). Tuple slicing returns a part of the tuple from the *start* index value to *stop* index value, which includes the *start* index value but excludes the *stop* index value. The *step* specifies the increment value to slice by and it is optional.

For the tuple *colors*, the positive and negative index breakdown is shown below.



For example,

1. >>> colors = ("v", "i", "b", "g", "y", "o", "r")
2. >>> colors
   ('v', 'i', 'b', 'g', 'y', 'o', 'r')

```
3. >>> colors[1:4]
   ('i', 'b', 'g')
4. >>> colors[:5]
   ('v', 'i', 'b', 'g', 'y')
5. >>> colors[3:]
   ('g', 'y', 'o', 'r')
6. >>> colors[:]
   ('v', 'i', 'b', 'g', 'y', 'o', 'r')
7. >>> colors[::]
   ('v', 'i', 'b', 'g', 'y', 'o', 'r')
8. >>> colors[1:5:2]
   ('i', 'g')
9. >>> colors[::2]
   ('v', 'b', 'y', 'r')
10. >>> colors[::-1]
    ('r', 'o', 'y', 'g', 'b', 'i', 'v')
11. >>> colors[-5:-2]
    ('b', 'g', 'y')
```

Built-In Functions Used on Tuples

Built-In Functions	Description
len()	The *len()* function returns the numbers of items in a tuple.
sum()	The *sum()* function returns the sum of numbers in the tuple.
sorted()	The *sorted()* function returns a sorted copy of the tuple as a list while leaving the original tuple untouched.

For example,

```
1. >>> years = (1987, 1985, 1981, 1996)
2. >>> len(years)
   4
3. >>> sum(years)
   7949
4. >>> sorted_years = sorted(years)
5. >>> sorted_years
   [1981, 1985, 1987, 1996]
```

Tuple Methods

We can get a list of all the methods associated with the tuple by passing thetuple function to dir().

```
1. >>> dir(tuple)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__
format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__
hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', 'count', 'index']
```

Tuple Methods	Syntax	Description
count()	tuple_name.count(item)	The *count()* method counts the number of times the item has occurred in the tuple and returns it.
index()	tuple_name.index(item)	The *index()* method searches for the given item from the start of the tuple and returns its index. If the value appears more than once, you will get the index of the first one. If the item is not present in the tuple, then *ValueError* is thrown by this method.

Note: Replace the word *"tuple_name"* mentioned in the syntax with your *actual tuple name* in your code.

For example,

```
1. >>> channels = ("ngc", "discovery", "animal_planet", "history", "ngc")
2. >>> channels.count("ngc")
   2
3. >>> channels.index("history")
   3
```

a. Tuple Packing and Unpacking

In packing, we put values into a new tuple while in unpacking we extract those values into a single variable.

The statement t = 12345, 54321, 'hello!' is an example of tuple packing.

```
>>> t = 12345, 54321, 'hello!'
>>> t
(12345, 54321, 'hello!')
```

The values 12345, 54321 and 'hello!' are packed together into a tuple.

The reverse operation of tuple packing is also possible. For example,

```
>>> x, y, z = t
>>> x
12345
>>> y
54321
>>> z
'hello!'
```

b. Traversing of Tuples

We can iterate through each item in tuples using for loop.

Example:

```
thistuple = ("apple", "banana", "cherry")
for x in thistuple:
  print(x)
```

Output:
apple
banana
cherry

c. Populating Tuples with Items

We can populate tuples with items using += operator and also by converting list items to tuple items.

Example: adding elements to tuple using += operator.

```
tuple_items = ()
total_items = int(input("Enter the total number of items: "))
for i in range(total_items):
    user_input = int(input("Enter a number: "))
    tuple_items += (user_input,)
print(f"Items added to tuple are {tuple_items}")
```

SBC Karkala

```
Output:
Enter the total number of items: 4
Enter a number: 4
Enter a number: 2
Enter a number: 5
Enter a number: 4
Items added to tuple are (4, 2, 5, 4)
```

Example: Converting list item into tuple

```
list_items = []
total_items = int(input("Enter the total number of items: "))
for i in range(total_items):
    item = input("Enter an item to add: ")
    list_items.append(item)
items_of_tuple = tuple(list_items)
print(f"Tuple items are {items_of_tuple}")
```

```
Output:
Enter the total number of items: 3
Enter an item to add: 4
Enter an item to add: 1
Enter an item to add: 6
Tuple items are ('4', '1', '6')
```

Sets

Python also includes a data type for sets. A set is an unordered collection with no duplicate items. Primary uses of sets include membership testing and eliminating duplicate entries. A set is a collection of unique items. Sets also support mathematical operations, such as union, intersection, difference, and symmetric difference.

Curly braces { } or the set() function can be used to create sets with a comma-separated list of items inside curly brackets { }. Note: to create an empty set you have to use set() and not { } as the latter creates an empty dictionary.

```
1. >>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
2. >>> print(basket)
   {'pear', 'orange', 'banana', 'apple'}
3. >>> 'orange' in basket
   True
4. >>> 'crabgrass' in basket
   False
5. >>> a = set('abracadabra')
6. >>> b = set('alacazam')
7. >>> a
   {'d', 'a', 'b', 'r', 'c'}
8. >>> b
   {'m', 'l', 'c', 'z', 'a'}
9. >>> a – b
   {'b', 'r', 'd'}
10. >>> a | b
    {'l', 'm', 'z', 'd', 'a', 'b', 'r', 'c'}
11. >>> a & b
    {'a', 'c'}
12. >>> a ^ b
    {'l', 'd', 'm', 'b', 'r', 'z'}
13. >>> len(basket)
    4
14. >>> sorted(basket)
    ['apple', 'banana', 'orange', 'pear']
```

SBC Karkala

Set Methods

We can get a list of all the methods associated with the set by passing the set function to dir().

```
>>> dir(set)
['__and__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__
format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__iand__', '__init__', '__
init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__',
'__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__',
'__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__sub-
classhook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard',
'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove',
'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
```

Various Set Methods

Set Methods	Syntax	Description
add()	set_name.add(*item*)	The *add()* method adds an *item* to the set *set_name*.
clear()	set_name.clear()	The *clear()* method removes all the items from the set *set_name*.
difference()	set_name.difference(**others*)	The *difference()* method returns a new set with items in the set *set_name* that are not in the *others* sets.
discard()	set_name.discard(item)	The *discard()* method removes an *item* from the set *set_name* if it is present.
intersection()	set_name.intersection(**others*)	The *intersection()* method returns a new set with items common to the set *set_name* and all *others* sets.
isdisjoint()	set_name.isdisjoint(*other*)	The *isdisjoint()* method returns True if the set *set_name* has no items in common with *other* set. Sets are disjoint if and only if their intersection is the empty set.
issubset()	set_name.issubset(*other*)	The *issubset()* method returns True if every item in the set *set_name* is in *other* set.
issuperset()	set_name.issuperset(*other*)	The *issuperset()* method returns True if every element in *other* set is in the set *set_name*.
pop()	set_name.pop()	The method *pop()* removes and returns an arbitrary item from the set set_name. It raises KeyError if the set is empty.
remove()	set_name.remove(*item*)	The method *remove()* removes an *item* from the set *set_name*. It raises KeyError if the *item* is not contained in the set.
symmetric_ difference()	set_name. symmetric_difference(other)	The method *symmetric_difference()* returns a new set with items in either the set or other but not both.
union()	set_name.union(**others*)	The method *union()* returns a new set with items from the set *set_name* and all *others* sets.
update()	set_name.update(**others*)	Update the set *set_name* by adding items from all *others* sets.

Note: Replace the words "*set_name*", "*other*" *and* "*others*" mentioned in the syntax with your *actual set names* in your code.

For example,

```
1. >>> european_flowers = {"sunflowers", "roses", "lavender", "tulips", "goldcrest"}
2. >>> american_flowers = {"roses", "tulips", "lilies", "daisies"}
3. >>> american_flowers.add("orchids")
4. >>> american_flowers.difference(european_flowers)
   {'lilies', 'orchids', 'daisies'}
```

```
5. >>> american_flowers.intersection(european_flowers)
   {'roses', 'tulips'}
6. >>> american_flowers.isdisjoint(european_flowers)
   False
7. >>> american_flowers.issuperset(european_flowers)
   False
8. >>> american_flowers.issubset(european_flowers)
   False
9. >>> american_flowers.symmetric_difference(european_flowers)
   {'lilies', 'orchids', 'daisies', 'goldcrest', 'sunflowers', 'lavender'}
10. >>> american_flowers.union(european_flowers)
    {'lilies', 'tulips', 'orchids', 'sunflowers', 'lavender', 'roses', 'goldcrest', 'daisies'}
11. >>> american_flowers.update(european_flowers)
12. >>> american_flowers
    {'lilies', 'tulips', 'orchids', 'sunflowers', 'lavender', 'roses', 'goldcrest', 'daisies'}
13. >>> american_flowers.discard("roses")
14. >>> american_flowers
    {'lilies', 'tulips', 'orchids', 'daisies'}
15. >>> european_flowers.pop()
    'tulips'
16. >>> american_flowers.clear()
17. >>> american_flowers
    set()
```

a. Traversing of Sets

We can iterate through each item in a set using a for loop.

Example: Program to Iterate Over Items in Sets Using for Loop

```
test_set = set("mumbai")
print(test_set)
for val in test_set:
   print(val)
```

```
Output:
{'u', 'b', 'i', 'a', 'm'}
u
b
i
a
m
```